

■ 강의명: CSCI89

■ 주차: Lecture 6

■ 교수명: UIUC Student

■ 목적: 본 문서는 CSCI89 Lecture 6 강의 내용을 바탕으로 작성되었습니다. 주요 내용은 다음과 같습니다:

- 퀴즈 복습: TF-IDF, 의미론적 드리프트, 코사인 유사도 계산.
- 문자 임베딩 (Character Embeddings): 단어 임베딩의 한계를 극복하고, 철자 오류나 형태론적으로 복잡한 언어를 처리하는 방법.
- 오토인코더 (Autoencoders): 데이터의 효율적인 표현 학습을 위한 비지도 학습 모델.
- 스택형 오토인코더 (Stacked Autoencoders): 여러 계층을 가진 오토인코더의 훈련 방법.
- 희소 오토인코더 (Sparse Autoencoders): 잠재 공간의 희소성을 강제하여 효율성을 높이는 방법 (L1 정규화, KL 발산).
- 변이형 오토인코더 (Variational Autoencoders, VAEs): 잠재 공간을 구조화하고 새로운 데이터를 생성하는 방법.

이 문서는 비전공자도 쉽게 이해할 수 있도록 상세한 설명과 예시를 포함합니다.

Contents

1 용어 정리	2
2 퀴즈 복습: 핵심 개념 이해	3
2.1 TF-IDF (Term Frequency-Inverse Document Frequency)	3
2.1.1 TF (Term Frequency): 단어 빈도	3
2.1.2 IDF (Inverse Document Frequency): 역문서 빈도	3
2.1.3 TF-IDF 계산 예시	4
2.2 정적 단어 임베딩의 한계: 의미론적 드리프트와 문맥 비민감성	5
2.2.1 의미론적 드리프트 (Semantic Drift)	5
2.2.2 문맥 비민감성 (Context Insensitivity)	5
2.3 코사인 유사도 (Cosine Similarity)	6
2.3.1 코사인 유사도 공식	6
2.3.2 코사인 유사도 계산 예시	6
3 문자 임베딩 (Character Embeddings)	8
3.1 문자 임베딩의 필요성	8
3.2 문자 임베딩의 작동 방식	8
3.3 하이브리드 접근 방식 (Hybrid Approaches)	8
3.4 문자 임베딩 모델 구축 예시 (TensorFlow Keras)	9
4 오토인코더 (Autoencoders)	11
4.1 오토인코더의 동기 부여	11

4.2	오토인코더의 구조	11
4.2.1	재구성 손실 (Reconstruction Loss)	11
4.2.2	언더컴플리트 오토인코더 (Undercomplete Autoencoder)	11
4.3	오토인코더의 활용	12
4.4	노이즈 제거 오토인코더 (Denoising Autoencoders)	12
5	스택형 오토인코더 (Stacked Autoencoders)	13
5.1	구조 및 훈련의 어려움	13
5.2	탐욕적 계층별 사전 훈련 (Greedy Layer-wise Pre-training)	13
5.3	스택형 오토인코더 예시 (MNIST/Fashion MNIST)	14
5.4	t-SNE (t-Distributed Stochastic Neighbor Embedding)를 이용한 시각화	15
5.4.1	t-SNE의 작동 원리	15
5.5	오토인코더를 이용한 분류기 사전 훈련	15
6	희소 오토인코더 (Sparse Autoencoders)	17
6.1	희소 오토인코더의 필요성	17
6.2	희소성 강제 방법	17
6.2.1	L1 활동 정규화 (L1 Activity Regularization)	17
6.2.2	쿨백-라이블러 (Kullback-Leibler, KL) 발산 정규화	17
7	변이형 오토인코더 (Variational Autoencoders, VAEs)	19
7.1	일반 오토인코더 잠재 공간의 문제점	19
7.2	변이형 오토인코더의 핵심 아이디어	19
7.3	VAE의 손실 함수	19
7.4	VAE의 장점	20
8	체크리스트	21
8.1	학습 점검표	21
8.2	문서 작성 점검표	21
9	FAQ (자주 묻는 질문)	23

1 용어 정리

다음 표는 본 문서에서 사용되는 주요 용어들을 쉽게 설명합니다.

용어	쉬운 설명	원어	비고
TF-IDF	특정 문서에서 단어가 얼마나 중요한지 나타내는 값	Term Frequency-Inverse Document Frequency	정보 검색 및 텍스트 마이닝에 사용
TF	특정 단어가 문서 내에서 얼마나 자주 나타나는지	Term Frequency	단어의 빈도
IDF	특정 단어가 전체 문서 집합에서 얼마나 희귀한지	Inverse Document Frequency	단어의 희귀성
의미론적 드리프트	단어의 의미가 시간이 지남에 따라 변하는 현상	Semantic Drift	정적 임베딩의 한계
문맥 비민감성	단어가 여러 의미를 가질 때 문맥을 구분하지 못하는 것	Context Insensitivity	정적 임베딩의 한계
코사인 유사도	두 벡터가 얼마나 유사한지 각도를 통해 측정	Cosine Similarity	임베딩 벡터 간 유사도 측정
문자 임베딩	단어가 아닌 문자를 벡터로 표현하는 기법	Character Embedding	OOV, 철자 오류, 형태론적 복잡성 해결
OOV	모델 훈련 시 보지 못했던 단어	Out-Of-Vocabulary	
오토인코더	입력 데이터를 압축하고 다시 복원하는 신경망	Autoencoder	비지도 학습, 차원 축소, 특징 학습
인코더	입력 데이터를 압축된 잠재 표현으로 변환하는 부분	Encoder	
디코더	압축된 잠재 표현을 원래 데이터로 복원하는 부분	Decoder	
잠재 공간	인코더가 생성한 압축된 데이터 표현 공간	Latent Space / Coding Space	
재구성 손실	원본 입력과 복원된 출력 간의 차이	Reconstruction Loss	오토인코더의 학습 목표
스택형 오토인코더	여러 은닉 계층을 가진 오토인코더	Stacked Autoencoder	심층 학습
희소 오토인코더	잠재 표현의 일부 뉴런만 활성화되도록 강제하는 오토인코더	Sparse Autoencoder	과적합 방지, 특징 추출
KL 발산	두 확률 분포 간의 차이를 측정하는 통계량	Kullback-Leibler Divergence	희소성 정규화, VAE에 사용
변이형 오토인코더	잠재 공간을 확률 분포로 모델링하여 생성 능력을 갖춘 오토인코더	Variational Autoencoder (VAE)	데이터 생성, 잠재 공간 구조화
t-SNE	고차원 데이터를 2차원 또는 3차원으로 시각화하는 기법	t-Distributed Stochastic Neighbor Embedding	시각화 도구, 차원 축소 아님

2 퀴즈 복습: 핵심 개념 이해

이번 섹션에서는 지난 강의에서 다룬 핵심 개념들을 퀴즈를 통해 복습하고, 각 개념의 중요성과 계산 방법을 상세히 설명합니다.

2.1 TF-IDF (Term Frequency-Inverse Document Frequency)

TF-IDF는 문서 내 단어의 중요도를 측정하는 통계적 척도입니다. 특정 단어가 문서에 얼마나 자주 나타나는지(TF)와 전체 코퍼스에서 얼마나 희귀한지(IDF)를 모두 고려합니다.

□ 핵심 정보

단순히 단어 빈도만으로는 문서 내 단어의 중요도를 정확히 파악하기 어렵습니다. 예를 들어, 'the'와 같은 불용어는 자주 나타나지만 문서의 의미를 크게 좌우하지 않습니다. 반면, 특정 전문 용어는 드물게 나타나지만 해당 문서의 핵심 주제를 나타낼 수 있습니다. TF-IDF는 이러한 점을 보완하여 단어의 실제 중요도를 반영합니다.

2.1.1 TF (Term Frequency): 단어 빈도

TF는 특정 단어가 주어진 문서 내에서 얼마나 자주 나타나는지를 측정합니다.

- **한 줄 핵심 요약:** 특정 문서에서 특정 단어가 얼마나 많이 등장하는가.
- **직관적 예시:** "사과 바나나 사과"라는 문서에서 '사과'는 2번 등장합니다.
- **기술적 설명:**

$$TF(t, d) = \frac{\text{문서 } d \text{ 내 단어 } t \text{의 등장 횟수}}{\text{문서 } d \text{ 내 전체 단어 수}}$$

여기서 t 는 단어(term), d 는 문서(document)를 의미합니다.

2.1.2 IDF (Inverse Document Frequency): 역문서 빈도

IDF는 특정 단어가 전체 코퍼스(문서 집합)에서 얼마나 희귀한지를 측정합니다. 희귀한 단어일수록 IDF 값이 높아져 중요도가 커집니다.

- **한 줄 핵심 요약:** 전체 문서들 중에서 특정 단어가 얼마나 드물게 등장하는가.
- **직관적 예시:** '사과'라는 단어가 100개의 문서 중 2개에만 있다면 매우 희귀하고 중요할 수 있습니다.
- **기술적 설명:**

$$IDF(t, D) = \log \left(\frac{\text{전체 문서 수 } (N)}{\text{단어 } t \text{를 포함하는 문서 수 } (df)} \right)$$

여기서 D 는 전체 코퍼스(corpus), N 은 전체 문서 수, df 는 단어 t 를 포함하는 문서의 수를 의미합니다. 로그 함수를 사용하는 이유는 단어 빈도수의 차이를 완화하고, 값이 너무 커지는 것을 방지하기 위함입니다.

□ 핵심 정보

강의에서 로그 함수의 밑(base)에 대한 질문이 있었습니다. 자연로그(ln, 밑이 e)를 사용하면 상용로그(log10, 밑이 10)를 사용하는 결과는 본질적으로 동일합니다. 이는 로그의 밑 변환 공식 $\log_a b = \frac{\ln b}{\ln a}$ 에 따라, 밑이 바뀌면 단순히 상수배가 되기 때문입니다. 최종적으로 임베딩 벡터를 정규화(normalize)

하는 경우, 이 상수배는 결과에 아무런 영향을 미치지 않습니다.

2.1.3 TF-IDF 계산 예시

다음은 주어진 코퍼스에서 'apple' 단어의 TF-IDF를 계산하는 예시입니다.

□ 예제

코퍼스 (전체 문서 집합):

- 문서 1: "apple banana apple"
- 문서 2: "banana orange"
- 문서 3: "apple orange banana"

계산 목표: 문서 1에서 'apple' 단어의 TF-IDF 값

1. TF (Term Frequency) 계산:

- 문서 1 내 'apple' 등장 횟수: 2회
- 문서 1 내 전체 단어 수: 3개 ('apple', 'banana', 'apple')
- $TF(\text{apple}, \text{문서 1}) = \frac{2}{3}$

2. IDF (Inverse Document Frequency) 계산:

- 전체 문서 수 (N): 3개
- 'apple'을 포함하는 문서 수 (df): 문서 1, 문서 3 (총 2개)
- $IDF(\text{apple}, \text{코퍼스}) = \log\left(\frac{3}{2}\right) = \log(1.5)$

3. TF-IDF 계산:

- $TF-IDF(\text{apple}, \text{문서 1}) = TF(\text{apple}, \text{문서 1}) \times IDF(\text{apple}, \text{코퍼스})$
- $TF-IDF(\text{apple}, \text{문서 1}) = \frac{2}{3} \times \log(1.5) \approx 0.6667 \times 0.4055 \approx 0.2703$

주의사항

Python의 `TfidfVectorizer`와 같은 라이브러리는 TF-IDF를 계산할 때 다양한 변형(예: 스무딩을 위한 +1 추가, 이중 정규화, 로그 밑 변경)을 사용합니다. 따라서 직접 계산한 값과 라이브러리 결과가 다를 수 있습니다. 이는 구현 방식의 차이일 뿐, 기본 원리는 동일합니다.

2.2 정적 단어 임베딩의 한계: 의미론적 드리프트와 문맥 비민감성

정적 단어 임베딩(Static Word Embeddings)은 한 번 학습되면 고정된 단어 벡터를 사용합니다. 이는 언어의 변화나 단어의 다의성을 반영하지 못하는 한계를 가집니다.

2.2.1 의미론적 드리프트 (Semantic Drift)

- **한 줄 핵심 요약:** 단어의 의미가 시간이 흐름에 따라 변하는 현상.
- **직관적 예시:** 과거의 'cool'은 '시원하다'는 의미였지만, 현재는 '멋지다'는 의미로도 널리 사용됩니다.
- **기술적 설명:** 정적 임베딩은 특정 시점의 코퍼스로 학습되므로, 10년 후 단어의 의미가 변하면 해당 단어의 임베딩 벡터는 더 이상 정확한 의미를 반영하지 못하게 됩니다. 이는 언어가 끊임없이 진화하기 때문에 발생하는 문제입니다.

2.2.2 문맥 비민감성 (Context Insensitivity)

- **한 줄 핵심 요약:** 단어가 여러 의미를 가질 때, 문맥에 따라 다른 의미를 구분하지 못하고 하나의 고정된 벡터를 사용하는 문제.
- **직관적 예시:** 'bank'는 '은행'과 '강둑'이라는 두 가지 의미를 가집니다. 정적 임베딩은 이 두 의미를 구분하지 못하고 'bank'에 대해 하나의 벡터만 제공합니다.
- **기술적 설명:** 정적 임베딩은 단어 하나당 하나의 고정된 벡터를 할당합니다. 따라서 다의어(polysemous words)의 경우, 문맥에 따라 달라지는 의미를 포착할 수 없습니다. 이는 현재 시점에서도 단어의 다양한 의미를 반영하지 못하는 문제입니다.

Table 1: 의미론적 드리프트와 문맥 비민감성 비교

특징	의미론적 드리프트	문맥 비민감성
시간 축	시간 경과에 따른 의미 변화	특정 시점 내 문맥에 따른 의미 변화
원인	언어의 진화	단어의 다의성
영향	오래된 임베딩의 유효성 감소	다의어 처리의 한계

2.3 코사인 유사도 (Cosine Similarity)

코사인 유사도는 두 벡터 간의 각도 코사인 값을 측정하여 벡터의 유사도를 판단하는 방법입니다. 각도가 작을수록(코사인 값이 1에 가까울수록) 두 벡터는 더 유사하다고 간주됩니다.

□ 핵심 정보

단어 임베딩에서 단어들은 고차원 공간의 벡터로 표현됩니다. 이때 두 단어의 의미적 유사성은 이들 벡터 간의 '거리'나 '각도'로 측정될 수 있습니다. 코사인 유사도는 벡터의 크기(길이)보다는 방향에 초점을 맞춰, 두 단어가 얼마나 유사한 문맥에서 사용되는지를 효과적으로 나타냅니다. 이는 임베딩 학습 시 손실 함수(loss function)가 내적(dot product)과 관련되어 있기 때문에 자연스럽게 유사도 척도로 사용됩니다.

2.3.1 코사인 유사도 공식

두 벡터 **A**와 **B** 사이의 코사인 유사도는 다음과 같이 정의됩니다.

$$\text{Cosine Similarity}(\mathbf{A}, \mathbf{B}) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \|\mathbf{B}\|} = \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \sqrt{\sum_{i=1}^n B_i^2}}$$

여기서 $\mathbf{A} \cdot \mathbf{B}$ 는 두 벡터의 내적(dot product)을, $\|\mathbf{A}\|$ 와 $\|\mathbf{B}\|$ 는 각 벡터의 유클리드 노름(Euclidean norm, 길이)을 의미합니다.

주의사항

두 벡터 중 하나라도 0 벡터(**0**)인 경우, 해당 벡터의 길이가 0이 되어 분모가 0이 됩니다. 이 경우 각도가 정의되지 않으므로, 코사인 유사도를 계산할 수 없습니다. 따라서 벡터의 길이가 0이 아님을 가정합니다.

2.3.2 코사인 유사도 계산 예시

다음은 'cat' 단어와 'dog', 'mouse' 단어 간의 코사인 유사도를 계산하여 어떤 단어가 'cat'과 가장 유사한지 판단하는 예시입니다.

□ 예제

주어진 단어 벡터:

- Cat: **C** = (2, 3)
- Dog: **D** = (5, 7)
- Mouse: **M** = (1, 2)

1. Cat과 Dog 간의 코사인 유사도:

- 내적: $\mathbf{C} \cdot \mathbf{D} = (2 \times 5) + (3 \times 7) = 10 + 21 = 31$
- Cat 벡터 길이: $\|\mathbf{C}\| = \sqrt{2^2 + 3^2} = \sqrt{4 + 9} = \sqrt{13} \approx 3.6056$
- Dog 벡터 길이: $\|\mathbf{D}\| = \sqrt{5^2 + 7^2} = \sqrt{25 + 49} = \sqrt{74} \approx 8.6023$
- 코사인 유사도: $\frac{31}{\sqrt{13} \times \sqrt{74}} = \frac{31}{\sqrt{962}} \approx \frac{31}{31.0161} \approx 0.99948$
- 강의에서 제시된 값: 0.9948 (계산 도구의 반올림 또는 내부 로직 차이로 인한 미세한 오차 가능성)

2. Cat과 Mouse 간의 코사인 유사도:

- 내적: $\mathbf{C} \cdot \mathbf{M} = (2 \times 1) + (3 \times 2) = 2 + 6 = 8$
- Cat 벡터 길이: $\|\mathbf{C}\| = \sqrt{13} \approx 3.6056$ (위와 동일)
- Mouse 벡터 길이: $\|\mathbf{M}\| = \sqrt{1^2 + 2^2} = \sqrt{1 + 4} = \sqrt{5} \approx 2.2361$
- 코사인 유사도: $\frac{8}{\sqrt{13} \times \sqrt{5}} = \frac{8}{\sqrt{65}} \approx \frac{8}{8.0623} \approx 0.99228$
- 강의에서 제시된 값: 0.9922 (계산 결과와 일치)

결론: Cat과 Dog의 유사도(약 0.9948)가 Cat과 Mouse의 유사도(약 0.9922)보다 높습니다. 따라서 'dog'이 'cat'에 더 가깝다고 할 수 있습니다. 이는 벡터의 길이가 아닌 방향(각도)을 기준으로 판단한 결과입니다.

3 문자 임베딩 (Character Embeddings)

이전까지는 단어(word)를 토큰으로 사용하여 임베딩을 생성했지만, 문자 임베딩은 텍스트를 개별 문자(character) 단위로 분해하여 임베딩을 생성하는 기법입니다.

3.1 문자 임베딩의 필요성

단어 임베딩은 강력하지만, 다음과 같은 한계가 있습니다.

- **OOV (Out-Of-Vocabulary) 문제:** 훈련 데이터에 없는 새로운 단어가 나타나면 해당 단어에 대한 임베딩을 생성할 수 없습니다. 이는 철자 오류(misspellings)가 있는 단어에도 해당됩니다.
- **형태론적 복잡성:** 터키어, 핀란드어와 같이 어미 변화가 매우 복잡하여 한 단어에 수십 가지 형태가 존재하는 언어의 경우, 단어 단위 토큰화는 어휘 크기를 폭발적으로 증가시킵니다.
- **접두사/접미사 처리:** 단어 임베딩은 'run', 'running', 'runner'를 별개의 단어로 취급합니다. 문자 임베딩은 이러한 형태론적 정보를 더 잘 포착할 수 있습니다.
- **철자 오류 교정 및 이름 개체 인식 (Named Entity Recognition, NER):** 문자 수준에서 텍스트를 분석하면 철자 오류를 감지하고 수정하거나, 훈련 시 보지 못한 고유 명사(이름, 장소 등)를 인식하는데 유리합니다.

□ 핵심 정보

문자 임베딩은 OOV 문제에 강하고, 형태론적 정보를 효과적으로 학습하며, 철자 오류나 새로운 단어에 대한 유연한 처리가 가능합니다. 이는 특히 형태론적으로 풍부한 언어나, 오타가 잦은 사용자 생성 텍스트 분석에 유용합니다.

3.2 문자 임베딩의 작동 방식

- **토큰화 (Tokenization):** 텍스트를 단어 대신 개별 문자로 분리합니다. 예를 들어, "hello"는 'h', 'e', 'l', 'l', 'o'로 분리됩니다.
- **임베딩 벡터 생성:** 각 문자는 고유한 임베딩 벡터로 변환됩니다. 이 벡터는 훈련 과정에서 학습됩니다.
- **시퀀스 처리:** 문장이나 단어는 문자 임베딩 벡터들의 시퀀스로 표현됩니다. 이후 RNN (Recurrent Neural Network)이나 CNN (Convolutional Neural Network)과 같은 시퀀스 모델을 사용하여 처리됩니다.

□ 핵심 정보

단어 임베딩의 경우 보통 100 300차원의 벡터를 사용하지만, 문자 임베딩은 어휘(vocabulary) 크기가 훨씬 작기 때문에 (알파벳 26자 + 숫자 + 특수문자 등) 8 20차원과 같이 더 낮은 차원의 벡터로도 충분히 좋은 성능을 낼 수 있습니다.

3.3 하이브리드 접근 방식 (Hybrid Approaches)

문자 임베딩과 단어 임베딩의 장점을 결합하기 위해 하이브리드 접근 방식이 자주 사용됩니다.

- 단어를 구성하는 문자들의 임베딩 시퀀스를 RNN/CNN으로 처리하여 단어 수준의 문자 임베딩을 생성합니다.

- 이 문자 기반 단어 임베딩과 기존의 단어 임베딩을 결합(예: 연결(concatenate))하여 최종 단어 표현을 만듭니다.
- 이렇게 하면 단어의 의미적 정보와 형태론적 정보를 모두 활용할 수 있습니다.

3.4 문자 임베딩 모델 구축 예시 (TensorFlow Keras)

다음은 TensorFlow Keras를 사용하여 문자 임베딩 기반의 신경망 모델을 구축하는 간단한 예시입니다.

□ 예제

1. 데이터 준비:

- 텍스트 데이터: ["hello world", "machine learning", "deep learning"]
- 레이블: [1, 0, 0] (예시 분류 문제)

2. 문자 수준 토큰화:

```

1 import tensorflow as tf
2 from tensorflow.keras.preprocessing.text import Tokenizer
3 from tensorflow.keras.preprocessing.sequence import pad_sequences
4 from tensorflow.keras.models import Sequential
5 from tensorflow.keras.layers import Embedding, LSTM, Dense
6 import numpy as np
7
8 texts = ["hello world", "machine learning", "deep learning"]
9 labels = np.array([1, 0, 0])
10
11 # character_level=로 True 문자수준토큰화설정
12 tokenizer = Tokenizer(char_level=True)
13 tokenizer.fit_on_texts(texts)
14
15 # 어휘크기계산패딩 ( 토큰을위해 +1)
16 vocab_size = len(tokenizer.word_index) + 1
17 print(f"문자 어휘크기 : {vocab_size}")
18 # 출력예시 : 문자어휘크기
19           기 : 16 (h, e, l, o, w, r, d, m, a, c, i, n, g, p, _ 공
20           백(), +1 for padding)
21
22 # 텍스트를시퀀스로변환
23 sequences = tokenizer.texts_to_sequences(texts)
24
25 # 시퀀스패딩가장 ( 긴시퀀스길이에맞춰서 0 채움)
26 max_len = max(len(s) for s in sequences)
27 padded_sequences = pad_sequences(sequences, maxlen=max_len, padding='post')
28 print(f"패딩된 시퀀스:\n{padded_sequences}")
29 # 출력예시 :
30 # [[ 1  2  3  3  4  5  6  7  0  0  0  0  0  0  0]
31 #  [ 8  9 10 11 12 13 14 15  0  0  0  0  0  0]
32 #  [15 16 17 18 19 20 21 22 23 24 25 26 27 28 29]]

```

Listing 1: 문자 수준 토큰화

- `tokenizer.word_index` : 'e' 1, 'l' 2

- `vocab_size + 1` (*padding*) (`0`) `.padding='post'` `0` .

3. 모델 구축:

```

1 embedding_dim = 8 # 문자임베딩차원단어보다 (작음)
2
3 model = Sequential([
4     Embedding(vocab_size, embedding_dim, input_length=max_len),
5     LSTM(16), # LSTM 레이어
6     Dense(1, activation='sigmoid') # 이진분류를위한출력레이어
7 ])
8
9 model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['
    accuracy'])
10 model.summary()
```

Listing 2: 문자 임베딩 모델 구축

- Embedding 레이어는 각 문자를 `embedding_dim` .
- LSTM (Long Short-Term Memory) 레이어는 시퀀스 데이터를 처리하고 장기 의존성을 학습합니다.
- Dense 레이어는 최종 분류를 수행합니다 (여기서는 이진 분류).

4. 모델 훈련:

```

1 model.fit(padded_sequences, labels, epochs=10, batch_size=2)
```

Listing 3: 모델 훈련

□ 핵심 정보

`tokenizer.word_index` `','` `'e'` `','` `'l'` `','` `'n'` . 빈도수 .

4 오토인코더 (Autoencoders)

오토인코더는 입력 데이터를 효율적인 저차원 표현(잠재 표현 또는 인코딩)으로 압축한 다음, 이 압축된 표현으로부터 원본 데이터를 최대한 가깝게 복원하도록 훈련되는 신경망입니다.

4.1 오토인코더의 동기 부여

- **효율적인 정보 표현:** 복잡한 데이터를 더 간결하고 의미 있는 형태로 저장하거나 전달하는 방법을 찾는 것이 목표입니다. 예를 들어, 긴 숫자 시퀀스 70, 68, 66, 64, 62, 60은 70에서 시작하여 2씩 감소하는 6개의 숫자로 훨씬 효율적으로 기억할 수 있습니다.
- **데이터의 구조 이해:** 오토인코더는 데이터 내재된 중요한 특징이나 패턴을 학습하여, 이를 통해 데이터의 본질적인 구조를 파악합니다.

□ 핵심 정보

오토인코더는 입력 데이터 $\mathbf{x}$ 를 받아 이를 잠재 표현 $\mathbf{c}$ 로 인코딩한 다음, 이 $\mathbf{c}$ 로부터 다시 원본과 유사한 $\hat{\mathbf{x}}$ 를 디코딩합니다. 이때 $\mathbf{x}$ 와 $\hat{\mathbf{x}}$ 의 차이(재구성 손실)를 최소화하도록 학습됩니다. 인코딩 과정에서 병목(bottleneck) 구조를 강제하여, 모델이 데이터의 가장 중요한 특징만을 압축하도록 만듭니다.

4.2 오토인코더의 구조

오토인코더는 크게 두 부분으로 구성됩니다.

1. **인코더 (Encoder):** 입력 데이터 $\mathbf{x}$ 를 받아 저차원의 잠재 표현 $\mathbf{c}$ (또는 코딩, 인코딩)로 변환합니다.
2. **디코더 (Decoder):** 잠재 표현 $\mathbf{c}$ 를 받아 원본 데이터 $\mathbf{x}$ 와 유사한 재구성된 데이터 $\hat{\mathbf{x}}$ 로 변환합니다.

Figure 1: 오토인코더의 기본 구조 (입력-인코더-병목-디코더-출력)

위 그림은 오토인코더의 기본 구조를 나타냅니다. 입력 데이터가 인코더를 통해 저차원의 병목(bottleneck) 계층으로 압축되고, 이 병목 계층의 표현이 디코더를 통해 다시 원본과 유사한 형태로 복원됩니다.

4.2.1 재구성 손실 (Reconstruction Loss)

오토인코더의 학습 목표는 입력 $\mathbf{x}$ 와 재구성된 출력 $\hat{\mathbf{x}}$ 사이의 차이를 최소화하는 것입니다. 이를 재구성 손실(Reconstruction Loss)이라고 합니다.

$$\text{Loss} = \sum_{k=1}^N (\mathbf{x}_k - \hat{\mathbf{x}}_k)^2 \quad (\text{L2 노름 예시})$$

이 손실 함수를 최소화함으로써, 인코더는 입력 데이터를 가장 효율적으로 압축하는 방법을 학습하게 됩니다.

4.2.2 언더컴플리트 오토인코더 (Undercomplete Autoencoder)

잠재 표현의 차원(병목 계층의 뉴런 수)이 입력 데이터의 차원보다 작은 경우를 언더컴플리트 오토인코더라고 합니다. 이는 모델이 입력 데이터의 중요한 특징만을 학습하도록 강제하여 차원 축소 효과를

얻습니다.

□ 핵심 정보

만약 오토인코더의 활성화 함수가 선형이고, 손실 함수가 L2 노름이라면, 이 오토인코더는 주성분 분석(PCA)과 매우 유사하게 작동합니다. PCA는 데이터의 분산을 가장 잘 설명하는 직교 축을 찾는 반면, 선형 오토인코더는 데이터를 가장 잘 재구성할 수 있는 저차원 공간을 학습합니다.

4.3 오토인코더의 활용

1. **차원 축소 (Dimensionality Reduction):** 훈련된 오토인코더에서 디코더 부분을 제거하고 인코더만 사용하여 고차원 데이터를 저차원의 잠재 표현으로 변환합니다. 이 잠재 표현은 다른 머신러닝 모델의 입력으로 사용될 수 있습니다.
2. **특징 학습 (Feature Learning):** 오토인코더는 데이터의 본질적인 특징을 비지도 방식으로 학습합니다. 이렇게 학습된 특징은 분류, 군집화 등 다양한 다운스트림 태스크에 활용될 수 있습니다.
3. **데이터 압축:** 잠재 표현 자체가 원본 데이터의 압축된 형태이므로, 데이터 저장 및 전송에 활용될 수 있습니다.

□ 핵심 정보

오토인코더는 입력 신호 자체를 재구성하는 것을 목표로 합니다. 반면, 단어 임베딩(예: Word2Vec)은 단어를 주변 문맥에 매핑하거나 분류하는 등 다른 태스크를 통해 단어 벡터를 학습합니다. 하지만 둘 다 고차원 데이터를 저차원 벡터로 표현한다는 공통점이 있습니다.

4.4 노이즈 제거 오토인코더 (Denoising Autoencoders)

노이즈 제거 오토인코더는 입력 데이터에 의도적으로 노이즈를 추가한 후, 이 노이즈가 제거된 원본 데이터를 복원하도록 훈련됩니다.

• 작동 방식:

1. 원본 데이터 x 에 인위적으로 노이즈를 추가하여 $\tilde{x}$ 를 만듭니다.
2. 오토인코더의 입력으로 $\tilde{x}$ 를 사용하고, 출력으로는 노이즈가 없는 원본 x 를 사용합니다.
3. 모델은 노이즈가 있는 입력에서 노이즈를 제거하고 원본을 복원하는 방법을 학습합니다.

• 장점:

- **강건한 특징 학습:** 모델이 노이즈에 강인한 특징을 학습하게 됩니다.
- **비지도 학습:** 레이블된 데이터가 필요 없습니다. 수많은 이미지나 텍스트 데이터에 노이즈를 추가하여 스스로 학습할 수 있습니다.
- **데이터 정제:** 훈련된 모델은 실제 노이즈가 포함된 데이터를 정제하는 데 사용될 수 있습니다.

□ 예제

텍스트 데이터의 경우, 문장에 의도적으로 오타를 삽입하거나 단어를 무작위로 삭제하는 방식으로 노이즈를 추가할 수 있습니다. 예를 들어, "Hello world"를 "Helo wrld"로 만든 후, 오토인코더가 이를 다시 "Hello world"로 복원하도록 학습시킵니다. 이렇게 훈련된 모델은 실제 오타가 포함된 문장을 교정하는 데 활용될 수 있습니다.

5 스택형 오토인코더 (Stacked Autoencoders)

스택형 오토인코더는 여러 개의 은닉 계층을 가진 심층 신경망 형태의 오토인코더입니다. 인코더와 디코더 모두 여러 계층으로 구성됩니다.

5.1 구조 및 훈련의 어려움

- **구조:** 입력 계층, 여러 은닉 계층(인코더), 병목 계층, 여러 은닉 계층(디코더), 출력 계층으로 구성됩니다. 일반적으로 인코더와 디코더는 계층 수와 뉴런 수가 대칭적인 형태를 가집니다.
- **심층 신경망 훈련의 어려움:** 심층 신경망은 기울기 소실(vanishing gradients) 또는 기울기 폭주(exploding gradients)와 같은 문제로 인해 훈련이 어렵습니다. 손실 함수의 표면이 매우 복잡하여 최적화가 쉽지 않습니다.

Figure 2: 스택형 오토인코더의 구조 예시 (784 → 100 → 30 → 100 → 784)

위 그림은 784차원 입력이 100차원, 30차원으로 압축된 후 다시 100차원, 784차원으로 복원되는 스택형 오토인코더의 예시를 보여줍니다. 30차원 계층이 병목 역할을 합니다.

5.2 탐욕적 계층별 사전 훈련 (Greedy Layer-wise Pre-training)

심층 스택형 오토인코더를 한 번에 훈련하는 대신, 각 계층을 개별적으로 사전 훈련하는 방법입니다. 이는 심층 신경망 훈련의 어려움을 완화하는 데 도움이 됩니다.

1. **1단계:** 첫 번째 오토인코더를 훈련합니다. 이 오토인코더는 원본 입력을 받아 첫 번째 은닉 계층의 크기(예: 784 → 100 → 784)로 압축 및 복원합니다.
2. **2단계:** 1단계에서 훈련된 인코더의 출력(100차원 잠재 표현)을 새로운 입력으로 사용하여 두 번째 오토인코더를 훈련합니다. 이 오토인코더는 100차원 입력을 받아 두 번째 은닉 계층의 크기(예: 100 → 30 → 100)로 압축 및 복원합니다. 이때 1단계의 인코더 가중치는 고정(freeze)할 수 있습니다.
3. **N단계:** 이 과정을 반복하여 모든 계층을 순차적으로 훈련합니다.
4. **미세 조정 (Fine-tuning):** 모든 계층이 사전 훈련된 후, 전체 스택형 오토인코더를 다시 한 번 훈련하여 모든 가중치를 미세 조정할 수 있습니다.

□ 핵심 정보

- **훈련 안정성:** 각 계층을 얇은 오토인코더로 개별 훈련하므로, 심층 신경망의 훈련 불안정성 문제를 완화합니다.
- **더 나은 초기화:** 사전 훈련을 통해 가중치가 무작위 초기화보다 더 의미 있는 값으로 초기화되어, 전체 네트워크의 최종 훈련 성능을 향상시킬 수 있습니다.

주의사항

최근에는 Adam과 같은 개선된 최적화 알고리즘, ReLU와 같은 활성화 함수, 배치 정규화(Batch Normalization), 잔차 연결(Residual Connections) 등 다양한 기술의 발전으로 인해 심층 신경망을 한 번에 훈련하는 것이 과거보다 훨씬 쉬워졌습니다. 따라서 계층별 사전 훈련의 필요성이 예전만큼 크지는 않지만, 여전히 특정 상황에서는 유용한 전략입니다.

5.3 스택형 오토인코더 예시 (MNIST/Fashion MNIST)

28x28 픽셀 이미지(784차원)를 스택형 오토인코더로 압축 및 복원하는 예시입니다.

□ 예제

모델 구조:

- 입력: 784차원 (28x28 픽셀 이미지)
- 인코더: 100개 뉴런 → 30개 뉴런 (병목 계층)
- 디코더: 100개 뉴런 → 784개 뉴런 (출력 계층)

훈련:

- 입력 데이터: $X_{train}(\quad)$
- 출력 데이터: $X_{train}(\quad, \quad)$
- 에포크: 10회

결과: 훈련된 모델은 784차원의 이미지를 30차원의 잠재 표현으로 압축한 후 다시 784차원으로 복원합니다.

- 원본 이미지: 첫 번째 행에 원본 이미지가 표시됩니다.
- 재구성 이미지: 두 번째 행에 오토인코더가 복원한 이미지가 표시됩니다.

결과를 보면, 30차원이라는 매우 낮은 차원으로 압축되었음에도 불구하고 원본 이미지의 형태(예: 신발, 가방 등)를 상당 부분 유지하고 있음을 확인할 수 있습니다. 이는 오토인코더가 이미지의 핵심 특징을 성공적으로 학습했음을 의미합니다.

5.4 t-SNE (t-Distributed Stochastic Neighbor Embedding)를 이용한 시각화

t-SNE는 고차원 데이터를 2차원 또는 3차원으로 시각화하는 데 사용되는 비선형 차원 축소 기법입니다.

□ 핵심 정보

t-SNE는 주로 시각화를 위해 사용됩니다. 오토인코더가 생성한 고차원 잠재 표현(예: 30차원 인코딩)이 실제로 의미 있는 군집을 형성하는지, 데이터의 구조를 잘 반영하는지를 눈으로 확인하는 데 매우 유용합니다. t-SNE는 모델의 입력으로 사용되는 차원 축소 기법이 아니라는 점이 중요합니다.

5.4.1 t-SNE의 작동 원리

- **이웃 보존:** t-SNE는 고차원 공간에서 가까운 점들은 저차원 공간에서도 가깝게, 멀리 떨어진 점들은 멀리 떨어지도록 매핑하려고 시도합니다. 특히 '지역적 이웃(local neighborhood)' 관계를 잘 보존하는데 중점을 둡니다.
- **확률적 접근:** 데이터 포인트 간의 유사도를 확률 분포로 모델링하고, 저차원 공간에서도 이 확률 분포를 최대한 유사하게 만들려고 합니다.
- **비결정론적:** t-SNE는 확률적 알고리즘이므로, 실행할 때마다 결과가 약간씩 달라질 수 있습니다.

□ 예제

스택형 오토인코더가 생성한 30차원 잠재 표현에 t-SNE를 적용하여 2차원 평면에 시각화하면, 유사한 이미지들(예: 모든 신발, 모든 티셔츠)이 서로 가까이 군집을 형성하는 것을 볼 수 있습니다.

- **군집 형성:** 신발에 해당하는 잠재 표현들은 한 곳에 모여 클러스터를 형성하고, 바지에 해당하는 것들은 다른 클러스터를 형성합니다.
- **의미 있는 잠재 공간:** 이는 오토인코더가 30차원 잠재 공간에서 이미지의 중요한 의미적 특징을 성공적으로 분리하여 표현했음을 의미합니다.

5.5 오토인코더를 이용한 분류기 사전 훈련

오토인코더는 레이블이 없는 대량의 데이터로 사전 훈련되어, 레이블이 부족한 분류 문제에서 강력한 성능을 발휘할 수 있습니다.

1. 1단계: 오토인코더 훈련 (비지도 학습):

- 레이블이 없는 대량의 데이터(예: 수백만 장의 이미지)를 사용하여 오토인코더를 훈련합니다.
- 이 과정에서 오토인코더는 데이터의 효율적인 저차원 잠재 표현(인코딩)을 학습합니다.

2. 2단계: 인코더 추출 및 분류기 구축 (지도 학습):

- 훈련된 오토인코더에서 디코더 부분을 제거하고 인코더 부분만 추출합니다.
- 이 인코더의 가중치는 고정(freeze)합니다.
- 인코더의 출력(저차원 잠재 표현)을 새로운 분류 모델의 입력으로 사용합니다.
- 레이블이 있는 소량의 데이터(예: 수천 장의 이미지)를 사용하여 이 분류 모델을 훈련합니다.

□ 핵심 정보

- **레이블 데이터 부족 문제 해결:** 고차원 입력(예: 784차원 이미지)에 직접 분류기를 훈련하려면 매우 많은 레이블 데이터가 필요합니다. 하지만 오토인코더를 통해 저차원 잠재 표현(예: 30차원)을 얻으면, 훨씬 적은 수의 레이블 데이터로도 효과적인 분류기를 훈련할 수 있습니다.
- **강력한 특징 추출:** 오토인코더가 비지도 방식으로 학습한 잠재 표현은 데이터의 핵심 특징을 잘 나타내므로, 분류 성능 향상에 기여합니다.

이 접근 방식은 분류기의 일부(특징 추출 부분)를 레이블 없이 사전 훈련하는 것으로 볼 수 있습니다.

6 희소 오토인코더 (Sparse Autoencoders)

일반적인 오토인코더는 병목 계층의 뉴런 수가 입력 차원보다 작을 때 차원 축소 효과를 얻습니다. 하지만 데이터가 매우 이질적(예: 숫자, 동물, 자동차 이미지가 섞여 있음)인 경우, 병목 계층의 뉴런 수가 충분히 작지 않으면 모든 뉴런이 항상 활성화되어 희소성이 사라질 수 있습니다. 희소 오토인코더는 이러한 문제를 해결하기 위해 잠재 표현의 희소성을 강제합니다.

6.1 희소 오토인코더의 필요성

- **이질적인 데이터 처리:** 만약 오토인코더가 다양한 종류의 데이터(예: 숫자, 동물, 자동차)를 처리해야 하고, 병목 계층의 뉴런 수가 충분히 크다면(예: 90개), 각 입력(숫자, 동물, 자동차)에 대해 모든 90개 뉴런이 활성화될 수 있습니다.
- **특징 분리:** 이렇게 되면 특정 뉴런 그룹이 특정 유형의 데이터(예: 숫자)에만 반응하도록 학습되지 않고, 모든 뉴런이 모든 유형의 데이터에 대해 조금씩 활성화되어 특징 분리가 어려워집니다.
- **과적합 방지 및 효율성:** 잠재 표현이 희소하다는 것은 각 입력에 대해 소수의 뉴런만 활성화된다는 의미입니다. 이는 모델의 과적합을 방지하고, 데이터의 핵심 특징을 더 명확하게 추출하는 데 도움이 됩니다.

□ 핵심 정보

희소 오토인코더의 목표는 병목 계층의 뉴런 중 **일부만** 활성화되도록 강제하는 것입니다. 예를 들어, 90개의 뉴런 중 10%인 9개만 활성화되도록 하여, 각 입력에 대해 특정 뉴런 그룹이 전문화되도록 유도합니다.

6.2 희소성 강제 방법

희소성을 강제하는 주요 방법은 손실 함수에 추가적인 정규화 항을 도입하는 것입니다.

6.2.1 L1 활동 정규화 (L1 Activity Regularization)

- **작동 방식:** 잠재 표현 $\mathbf{c}$ 의 각 요소(C_i)의 절댓값 합을 손실 함수에 추가합니다.

$$\text{Loss} = \text{Reconstruction Loss} + \lambda \sum_{i=1}^N |C_i|$$

여기서 λ 는 정규화 강도를 조절하는 하이퍼파라미터입니다.

- **효과:** L1 정규화는 가중치를 0으로 만드는 경향이 있으므로, 잠재 표현의 일부 뉴런이 완전히 비활성화(0 값)되도록 유도합니다. 이는 희소성을 직접적으로 증가시킵니다.

6.2.2 쿨백-라이블러 (Kullback-Leibler, KL) 발산 정규화

KL 발산은 두 확률 분포 간의 차이를 측정하는 통계량입니다. 희소 오토인코더에서는 잠재 계층의 실제 평균 활성화 분포를 우리가 설정한 목표 희소성 분포에 가깝게 만들도록 손실 함수에 추가됩니다.

- **목표 희소성 설정:** 잠재 계층의 뉴런들이 평균적으로 얼마나 활성화되기를 원하는지 목표 비율 P 를 설정합니다(예: $P = 0.1$ 은 10%의 뉴런만 활성화되기를 원함).

- **실제 활성화 측정:** 훈련 중 잠재 계층 뉴런들의 실제 평균 활성화 비율 $\hat{P}$ 를 계산합니다.
- **KL 발산 추가:** 손실 함수에 P 와 $\hat{P}$ 사이의 KL 발산 항을 추가합니다.

$$\text{Loss} = \text{Reconstruction Loss} + \lambda \cdot D_{KL}(P||\hat{P})$$

여기서 $D_{KL}(P||\hat{P})$ 는 다음과 같이 정의됩니다 (이진 교차 엔트로피 기반):

$$D_{KL}(P||\hat{P}) = P \log \left(\frac{P}{\hat{P}} \right) + (1 - P) \log \left(\frac{1 - P}{1 - \hat{P}} \right)$$

- **효과:** KL 발산은 $\hat{P}$ 가 P 에 가까워지도록 강제합니다. 만약 $\hat{P}$ 가 P 보다 너무 크거나 작으면 KL 발산 값이 커져 손실이 증가하고, 모델은 $\hat{P}$ 를 P 에 맞추기 위해 뉴런 활성화를 조절하게 됩니다. 이는 L1 정규화보다 더 정교하게 목표 희소성을 제어할 수 있습니다.

□ 핵심 정보

우리는 "잠재 계층 뉴런의 10%만 활성화되도록 해!"라는 목표($P = 0.1$)를 설정합니다. 모델이 훈련되면서 실제 활성화되는 비율($\hat{P}$)이 계산됩니다. 만약 $\hat{P}$ 가 10%와 다르면, KL 발산 항이 손실을 증가시키고, 모델은 $\hat{P}$ 를 10%에 가깝게 만들기 위해 뉴런의 활성화를 조절합니다. 재구성 손실과 KL 발산 항은 서로 경쟁하며, 모델은 두 손실을 모두 최소화하는 방향으로 학습됩니다.

7 변이형 오토인코더 (Variational Autoencoders, VAEs)

일반적인 오토인코더의 잠재 공간은 '비구조적(unstructured)'입니다. 즉, 잠재 공간의 한 지점에서 아주 조금만 이동해도 완전히 다른, 심지어 무의미한 데이터가 재구성될 수 있습니다. 변이형 오토인코더(VAE)는 이러한 문제를 해결하고, 잠재 공간을 '구조화'하여 새로운 데이터를 생성할 수 있도록 설계되었습니다.

7.1 일반 오토인코더 잠재 공간의 문제점

- **불연속성:** 일반 오토인코더는 입력 데이터를 잠재 공간의 특정 '점'으로 매핑합니다. 이 점들 사이의 공간(틈새)은 훈련 데이터에 해당하지 않으므로, 이 틈새에 있는 잠재 표현으로부터 재구성된 데이터는 의미가 없거나 이상할 수 있습니다.
- **생성 능력 부족:** 잠재 공간이 구조화되어 있지 않기 때문에, 임의의 잠재 벡터를 샘플링하여 의미 있는 새로운 데이터를 생성하기 어렵습니다.

7.2 변이형 오토인코더의 핵심 아이디어

VAE는 입력 데이터를 잠재 공간의 특정 '점'으로 매핑하는 대신, '확률 분포'로 매핑합니다.

- **인코더의 출력:** VAE의 인코더는 잠재 공간의 각 차원에 대해 평균(μ)과 분산(σ^2)을 출력합니다. 즉, 입력 데이터는 잠재 공간에서 가우시안 분포(정규 분포)로 표현됩니다.
- **잠재 벡터 샘플링:** 이 분포($N(\mu, \sigma^2)$)에서 잠재 벡터 $\mathbf{z}$ 를 샘플링합니다. 이때 $\mathbf{z} = \mu + \sigma \cdot \epsilon$ 공식을 사용하는데, ϵ 은 표준 정규 분포($N(0, 1)$)에서 샘플링된 노이즈입니다. 이를 '재매개변수화 트릭(reparameterization trick)'이라고 합니다.
- **디코더:** 샘플링된 $\mathbf{z}$ 를 디코더에 입력하여 원본 데이터를 재구성합니다.

Figure 3: 변이형 오토인코더의 기본 구조 (인코더가 평균과 분산을 출력)

위 그림은 VAE의 구조를 보여줍니다. 인코더는 입력으로부터 잠재 공간의 평균(μ)과 로그 분산($\log \sigma^2$)을 예측하고, 이로부터 잠재 벡터 $\mathbf{z}$ 를 샘플링합니다. 디코더는 $\mathbf{z}$ 로부터 원본을 재구성합니다.

7.3 VAE의 손실 함수

VAE의 손실 함수는 두 가지 항으로 구성됩니다.

1. **재구성 손실 (Reconstruction Loss):** 일반 오토인코더와 동일하게, 원본 입력과 재구성된 출력 간의 차이를 최소화합니다. 이는 모델이 데이터를 정확하게 복원하도록 합니다.
2. **KL 발산 정규화 (KL Divergence Regularization):** 인코더가 출력하는 잠재 분포($N(\mu, \sigma^2)$)가 표준 정규 분포($N(0, 1)$)와 얼마나 다른지를 측정하는 KL 발산 항을 추가합니다.

$$\text{Loss} = \text{Reconstruction Loss} + D_{KL}(N(\mu, \sigma^2) || N(0, 1))$$

이 KL 발산 항은 다음과 같이 계산됩니다:

$$D_{KL}(N(\mu, \sigma^2) || N(0, 1)) = \frac{1}{2} \sum_{i=1}^D (\sigma_i^2 + \mu_i^2 - \log(\sigma_i^2) - 1)$$

여기서 D 는 잠재 공간의 차원입니다.

□ 핵심 정보

- **잠재 공간 구조화:** KL 발산 항은 인코더가 학습하는 잠재 분포를 표준 정규 분포에 가깝게 만듭니다. 이는 잠재 공간을 **연속적이고 부드럽게(continuous and smooth)** 만듭니다. 즉, 잠재 공간의 어떤 지점을 샘플링하더라도 의미 있는 데이터가 생성될 가능성이 높아집니다.
- **분산 붕괴 방지:** 만약 KL 발산 항이 없다면, 모델은 재구성 손실만 최소화하기 위해 σ 를 0에 가깝게 만들려고 할 것입니다. 이는 VAE를 일반 오토인코더처럼 만들고, 잠재 공간의 확률적 특성을 잃게 합니다. KL 발산 항은 σ 가 0으로 수축하는 것을 방지하고, 충분한 분산을 유지하도록 강제합니다.
- **평균을 0으로:** 또한, μ 가 0에 가깝도록 유도하여 잠재 공간이 원점을 중심으로 분포하도록 만듭니다.

7.4 VAE의 장점

- **생성 능력:** 잠재 공간이 표준 정규 분포를 따르도록 학습되므로, 표준 정규 분포에서 임의의 잠재 벡터 z 를 샘플링하여 디코더에 입력하면 **새로운, 그럴듯한 데이터**를 생성할 수 있습니다.
- **연속적인 잠재 공간:** 잠재 공간에서 한 지점에서 다른 지점으로 이동하면, 재구성된 데이터도 부드럽게 변화합니다. 예를 들어, '티셔츠'에 해당하는 잠재 벡터에서 '바지'에 해당하는 잠재 벡터로 이동하면, 이미지가 점진적으로 티셔츠에서 바지로 변형되는 것을 관찰할 수 있습니다.
- **이미지 편집:** 잠재 공간의 특정 차원이 데이터의 특정 속성(예: 이미지의 색상, 객체의 크기)과 연관되도록 학습될 수 있으며, 이를 조작하여 이미지 편집에 활용할 수 있습니다.

주의사항

만약 VAE 훈련 시 σ 에 대한 제한이 없다면, 모델은 재구성 손실만 최소화하기 위해 σ 를 0으로 만들려고 할 것입니다. 이는 노이즈를 추가하지 않아 재구성이 쉬워지기 때문입니다. KL 발산 항은 σ 가 0으로 수축하는 것을 방지하고, 잠재 공간에 의미 있는 분산을 유지하도록 강제하는 중요한 역할을 합니다.

8 체크리스트

본 문서를 학습하거나 작성할 때 활용할 수 있는 점검표입니다.

8.1 학습 점검표

- TF-IDF의 개념과 계산 방법을 이해했는가?
- 정적 임베딩의 한계(의미론적 드리프트, 문맥 비민감성)를 구분하고 설명할 수 있는가?
- 코사인 유사도 계산 방법을 이해하고 직접 계산할 수 있는가?
- 문자 임베딩이 필요한 이유와 작동 방식을 설명할 수 있는가?
- 오토인코더의 기본 구조와 재구성 손실의 의미를 이해했는가?
- 노이즈 제거 오토인코더의 목적과 작동 방식을 설명할 수 있는가?
- 스택형 오토인코더의 훈련 어려움과 계층별 사전 훈련 방법을 이해했는가?
- t-SNE가 시각화 도구임을 이해하고 그 역할을 설명할 수 있는가?
- 희소 오토인코더가 필요한 이유와 L1 정규화, KL 발산 정규화의 차이를 설명할 수 있는가?
- 변이형 오토인코더가 일반 오토인코더와 다른 점, 그리고 생성 능력을 갖는 이유를 설명할 수 있는가?
- VAE의 KL 발산 항이 잠재 공간을 구조화하고 분산 붕괴를 막는 역할을 이해했는가?

8.2 문서 작성 점검표

- 제공된 강의 자료의 모든 내용을 빠짐없이 포함했는가?
- 출처, 파일명, 페이지 언급을 금지했는가?
- LaTeX 소스 전체를 제공했는가?
- 한국어로 작성되었으며, 전문용어는 쉬운 풀이와 원어 병기를 병행했는가?
- 문장은 짧고 명확하며, 단락은 4-6줄 내외로 유지되었는가?
- 나열은 불릿/번호 목록, 소제목, 박스, 공백을 적극 활용했는가?
- A4 규격에 맞는 레이아웃(여백, 줄간격, 단락 설정)을 준수했는가?
- 헤더/푸터(문서명, 날짜, 쪽수)를 `fancyhdr`로 표시했는가?
- 요약, 주의, 예시 박스를 `tcolorbox`로 구성했는가?
- 표와 그림에는 캡션과 라벨을 필수로 포함하고, 설명과 가까이 배치했는가?
- 모든 표는 A4 폭(160mm)에 맞추고 또는 `adjustbox`를 사용했는가?
- 코드 예시는 `listings`를 사용하고 `breaklines=true`, 행번호, 캡션/라벨을 포함했는가?
- 각 섹션 시작 전

를 사용하여 페이지 경계를 명확히 했는가?

- 초심자도 완전히 이해할 수 있도록 '왜 이런 개념이 필요한가'부터 설명했는가?
- 공식/절차/명령문에는 반드시 최소 1개의 예시를 붙였는가?
- 모를 수 있는 포인트를 예측하여 미리 설명하고, 오해 가능성을 고려했는가?
- 한 개념을 '한 줄 핵심 요약', '직관적 예시/비유', '기술적 정확한 설명'의 세 단계로 설명했는가?
- 문맥 의존 표현 없이 문장 자체로 의미가 통하도록 '단락 독립성'을 확보했는가?
- 필요한 경우 '개념 간 비교표'를 추가했는가?
- LaTeX 안전 가드레일(특수문자 이스케이프, 백틱 금지 등)을 준수했는가?

9 FAQ (자주 묻는 질문)

□ 핵심 요약

Q1: TF-IDF에서 로그 함수의 밑이 중요하지 않다는 게 무슨 의미인가요? A1: 로그 함수의 밑이 e 든 10 이든, 밑이 바뀌면 IDF 값 전체에 단순히 상수배가 됩니다. 예를 들어 $\log_{10} x = \frac{\ln x}{\ln 10}$ 이므로, $\ln 10$ 이라는 상수가 곱해지는 것과 같습니다. 최종적으로 임베딩 벡터를 정규화(normalize)하면 이 상수배는 사라지므로, 최종 결과에 영향을 미치지 않습니다.

Q2: 문자 임베딩이 단어 임베딩보다 항상 좋은가요? A2: 항상 그렇지는 않습니다. 문자 임베딩은 OOV 문제나 형태론적으로 복잡한 언어에 강하지만, 단어 임베딩은 단어 수준의 의미론적 정보를 더 직접적으로 포착할 수 있습니다. 일반적으로는 두 가지를 결합한 하이브리드 접근 방식이 가장 좋은 성능을 보이는 경우가 많습니다.

Q3: 오토인코더의 잠재 공간 차원을 얼마나 작게 설정해야 하나요? A3: 잠재 공간의 차원(병목 계층의 뉴런 수)은 데이터의 복잡성과 원하는 압축률에 따라 달라집니다. 너무 작으면 정보 손실이 커지고, 너무 크면 압축 효과가 줄어들고 과적합 위험이 있습니다. 일반적으로는 실험을 통해 최적의 차원을 찾습니다.

Q4: 스택형 오토인코더를 계층별로 사전 훈련하는 것이 요즘도 필요한가요? A4: 과거에는 심층 신경망 훈련의 어려움 때문에 필수적이었지만, 최근에는 Adam과 같은 개선된 최적화 기법, 배치 정규화, 잔차 연결 등 다양한 기술 덕분에 한 번에 훈련하는 것이 더 일반적입니다. 하지만 특정 복잡한 모델이나 데이터셋에서는 여전히 유용한 전략이 될 수 있습니다.

Q5: 희소 오토인코더에서 L1 정규화와 KL 발산 정규화 중 어떤 것을 사용해야 하나요? A5: L1 정규화는 단순히 활성화를 0으로 만드는 경향이 있어 희소성을 유도합니다. KL 발산 정규화는 특정 목표 활성화 비율(P)을 설정하고, 실제 활성화 비율($\hat{P}$)이 이 목표에 가깝도록 더 정교하게 제어합니다. KL 발산이 더 세밀한 제어가 가능하며, VAE와 같은 생성 모델에서도 활용됩니다.

Q6: VAE의 잠재 공간이 '구조화'되었다는 것은 정확히 무엇을 의미하나요? A6: 일반 오토인코더의 잠재 공간은 듬성듬성한 점들로 이루어져 있어, 점들 사이의 공간은 의미가 없습니다. VAE는 잠재 공간을 연속적인 확률 분포로 채워 넣기 때문에, 잠재 공간의 어떤 지점을 선택하더라도 그럴듯한 데이터가 생성됩니다. 이는 잠재 공간에서 부드럽게 이동하며 데이터가 점진적으로 변형되는 것을 관찰할 수 있다는 의미입니다.